Decision Point for Selecting the Right API Mediation Technology

31 July 2024 - ID G00816358 - 53 min read

By: Steve Deng

Initiatives:Software Architecture and Integration for Technical Professionals; Adopt Modern Architectures and Technologies; Build and Deliver Software to Drive Business Results

API mediation technologies have diversified. Application and integration architects should use this decision framework to select the right mediation technologies, among enterprise API gateway, lightweight gateway, ingress gateway, and service mesh, for their multigrained services and APIs.

This research note is restricted to the personal use of .

Overview

Key Findings

- API management, API gateways and service mesh play a critical role to meet emerging requirements for serverless and managed container systems. This has led to overlapping and complementary features and capabilities in transformation, traffic management, security and observability.
- API gateways come in many forms, ranging from centralized enterprise API gateways to more focused lightweight gateways or ingress gateways dedicated to a group of cohesive services or domain APIs. More choices enable more optimized solutions but increase the risk of analysis paralysis.
- Ingress gateways have emerged to combine API mediation with the ingress controller or gateway API pattern to secure, route and monitor external traffic in container orchestration platforms.
- Lightweight API gateways are emerging as the chosen API gateway for internal API management (APIM) in cloud-native environments because of their smaller footprint, declarative configuration and DevOps readiness.
- Many organizations overbuy APIM products, leading to underutilization, delayed deployment and increased operational complexity and overheads. Conversely, organizations that underbuy or underutilize these products struggle to maintain visibility and control on inadequately secured shadow APIs.

Recommendations

For successful API mediation, application and integration architects should take the following actions:

- Deploy a combination of API mediation technologies because the requirements for north-south and east-west communication differ considerably. Optimize API governance by deploying the right technology for each communication path in your architecture stack.
- Optimize protection, visibility and developer engagement by choosing an APIM platform, including API gateways as the preferred API mediation technology for RESTful APIs published or shared internally or externally.

This research note is restricted to the personal use of .

- Protect and manage containerized services by using an ingress gateway to apply API-specific policies to requests from clients outside the container infrastructure. An ingress gateway helps define and secure the boundary for services deployed in a container orchestration platform.
- Implement robust service-to-service communication in container management clusters (such as Kubernetes) by using a service mesh to provide traffic management, observability and security policy enforcement. Ensure the benefits from a service mesh outweigh the increased resource overhead and operational complexity.

Decision Point Question

Which mediation technologies should I select to protect and manage my APIs and services communication?

API mediation based on API gateways remains the most common solution to secure, govern and monitor REST endpoints for external, private and internal consumption. However, wider adoption of mesh application and service architecture (MASA) and multigrained services (which include macro, mini and microservices) has led to rapid innovations in container orchestration platforms (e.g., Kubernetes) and service meshes (such as Istio).

The market for APIM is evolving as well. Incumbent APIM vendors are adding support for event-driven architecture (EDA), service mesh and GraphQL. Startups have introduced API gateway capabilities integrated with service meshes and container orchestration platforms to secure and manage the handoff from north-south to east-west traffic. Technical professionals defining an API and service communication architecture face the challenge of choosing the right API mediation technologies for consistent governance across APIs and services in a diverse and distributed enterprise application environment.

This research note is restricted to the personal use of .

Decision Overview

When selecting an API mediation technology, you must balance a variety of functional and nonfunctional criteria to make an optimal decision. This research's evaluation criteria does not consider constraints such as skills, cost, procurement time and deployment effort when it comes to implementing a new API mediation technology for the first time. These factors would distort the selection criteria if they were included because the first-time costs for a solution are dramatically higher than subsequent implementations.

This decision tool will help you select the right API mediation technologies from the following options:

Enterprise API Gateway

An enterprise API gateway is typically located on your network perimeter to secure inbound and outbound API traffic. It is best-suited for external-facing, productized and published APIs with monetization potential. But it can be used for managing internal APIs supporting cross-domain integration and shared services within your primary data center or cloud infrastructure. These gateways are often centrally managed by the integration/middleware team or security organization and may be part of a broader APIM solution, including API life cycle management and an API developer portal.

Examples of enterprise API gateways are:

- Axway Amplify Platform
- Google's Apigee and Apigee Hybrid
- IBM API Connect
- Microsoft Azure API Management
- Salesforce MuleSoft API Gateway

Enterprise API gateways that are lightweight and deployable by customers, such as Kong Gateway, Gravitee API Gateway and Azure self-hosted gateway can also be used as lightweight gateways.

Lightweight API Gateway

This research note is restricted to the personal use of .

Previously known as a “microgateway,” a lightweight API gateway is designed to be distributed and deployed close to individual services. Characteristics of a lightweight gateway include:

- Self-contained, with a small memory and process footprint
- Containerized
- Ready for continuous integration/continuous delivery (CI/CD)
- Dynamic
- Declaratively configured

Unlike enterprise API gateways, which are often centrally managed as shared middleware, a lightweight gateway can be individually deployed, configured and managed by a development team to meet the application’s needs.

Examples of lightweight gateways include:

- Apache APISIX
- Solo.io Gloo Gateway
- KrakenD API Gateway
- Kong Gateway
- Microsoft Azure self-hosted gateway
- MuleSoft Flex Gateway
- Spring Cloud Gateway
- Tyk Gateway

Many lightweight API gateways can also be used as enterprise API gateways.

Ingress Gateway

This research note is restricted to the personal use of .

An ingress gateway is part of the organization's container environment. It is a component that functions as the network communications gatekeeper for a container orchestration cluster or service mesh. All inbound traffic to services running inside the cluster must pass through the ingress gateway first. As a result, it becomes an ideal policy enforcement point (PEP) for protecting APIs implemented by the services deployed to the cluster or service mesh.

In the context of this document, ingress gateway collectively refers to a class of components that implement Kubernetes ingress functionality. They can use a variety of configuration models, including Kubernetes Ingress Controller, Kubernetes Gateway API and Istio Ingress Gateway.

Examples of ingress gateways include:

- Contour
- Ambassador Labs Emissary-Ingress
- Envoy Gateway
- Istio Ingress Gateway
- Solo.io Gloo Gateway
- Traefik Hub
- HashiCorp/IBM Consult API gateway
- Kubernetes Ingress
- NGINX Ingress Controller

Service Mesh

A service mesh mediates interservice (east-west) communication in an application domain. The service mesh abstracts away the cross-cutting concerns relating to security and traffic management, while enabling observability for the developers. The service mesh component implements these capabilities with consistency and a high degree of reliability. Popular service mesh implementations include:

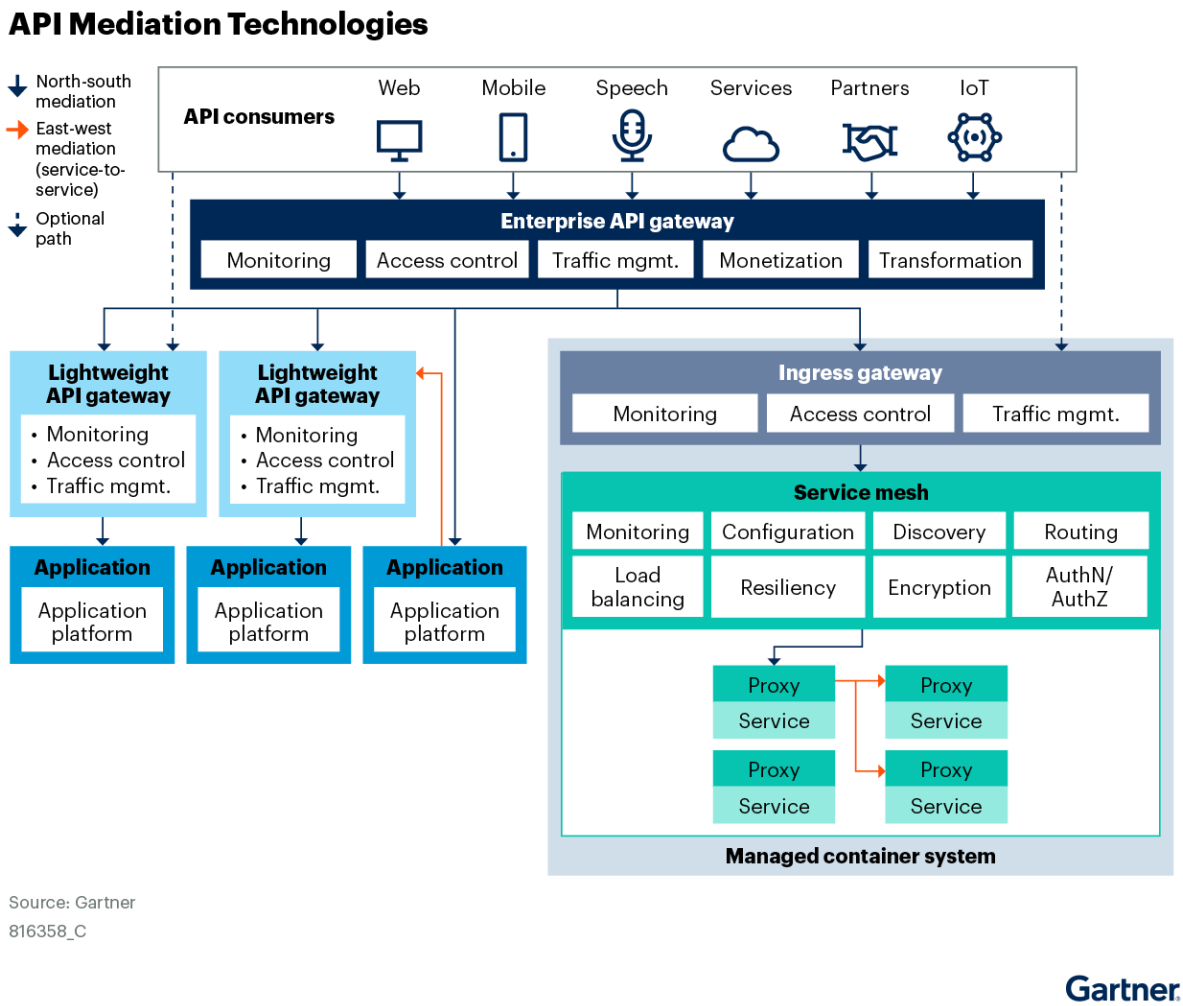
- Amazon Web Services (AWS) App Mesh

This research note is restricted to the personal use of .

- Google Cloud Service Mesh
- Istio
- Buoyant Linkerd
- HashiCorp Consul

These mediation technologies are implemented to sustain consistent management over diverse APIs and services workloads running across heterogeneous infrastructure. Each class of mediation technology has been designed to target specific use cases, workloads, architecture patterns or deployment topologies, as shown in Figure 1.

Figure 1: API Mediation Technologies



This research note is restricted to the personal use of .

A common architecture pattern uses a combination of mediation technologies. For example, use an enterprise API gateway to enforce enterprise-level security mediation policy protections. Then use a lightweight API gateway and an ingress gateway for application-specific, fine-grained policies, as well as a service mesh for east-west service and microservice communication within a domain. Technical professionals evaluating and selecting mediation technology should align their requirements and constraints to each mediation technology's optimal place of action. Organizations with diverse API and service implementations should select a combination of mediation technologies to comprehensively meet all API and service networking needs.

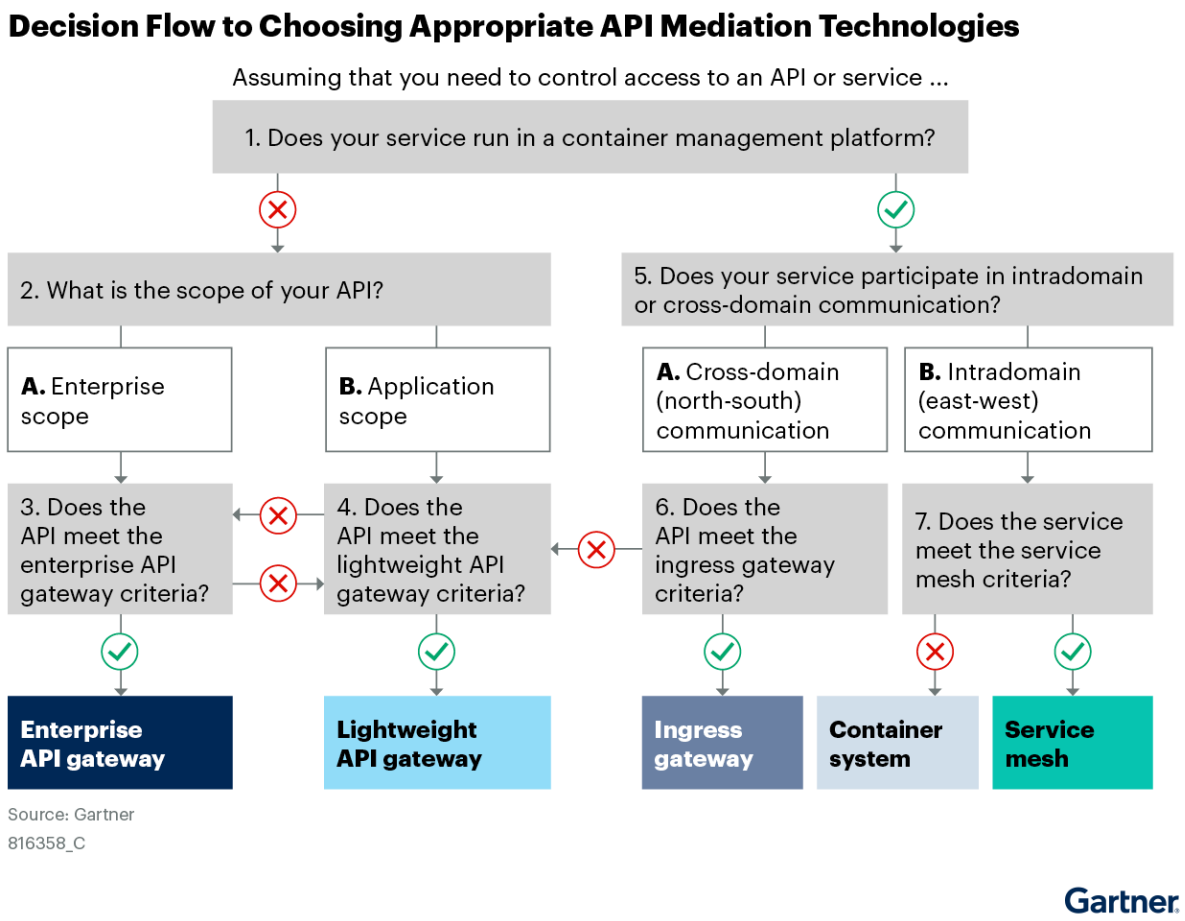
Decision Tool

The decision flow chart (see Figure 2) is designed to help you choose the appropriate API mediation technologies. It captures the most commonly observed constraints, variables and patterns among Gartner clients. Other factors that may be unique to your organization are outside the scope of the decision flow.

Choosing API mediation is a recurring decision because you need to choose mediation for each service you build. Sometimes it is obvious where it should be published/exposed, but the decision tool can help clarify this as teams deliver new APIs for different scenarios. The tool not only helps you choose what to deploy, it's also useful for choosing what to use from what has already been deployed.

This research note is restricted to the personal use of .

Figure 2: Decision Flow to Choosing Appropriate API Mediation Technologies



The decision flow begins with the assumption that you have a service and need to control access to its APIs. If the "inner" API exposed by your service is not readily consumed by all clients, you must define and publish a consumer-centric version of the API (outer API) for specific use cases. When multiple outer APIs are involved, revisit the flow for every new outer API definition.

The decision flow comprises the following questions to help you select the appropriate mediation technology:

- Step 1: Does your service run in a container management platform?
- Step 2: What is the scope of the API exposed by your service?
- Step 3: Does the API meet the enterprise API gateway criteria?
- Step 4: Does the API meet the lightweight API gateway criteria?

This research note is restricted to the personal use of .

- Step 5: Does your service participate in intradomain or cross-domain communication?
- Step 6: Does the API meet the ingress gateway criteria?
- Step 7: Does the service meet the service mesh criteria?

Step 1: Does Your Service Run in a Container Management Platform?

You must select the API mediation technology that is better integrated with your service's runtime infrastructure to improve performance and agility while reducing complexity.

If your service is containerized and runs in a container management platform such as Kubernetes, then your answer is "yes." You should proceed to **Step 5** to consider an ingress gateway and/or service mesh.

If your service is not containerized, then you should choose between an enterprise API gateway or lightweight API gateway. Proceed to **Step 2** of the decision flow.

In the case that the service is not containerized, your service implementation may be one of the following:

- **Virtual machine (VM) per service instance** — The service is deployed independently in a stand-alone VM (without assistance from a service mesh).
- **Enterprise app server including clustered app servers** — The service is deployed in an enterprise app server or a cluster of app servers, such as Oracle WebLogic Server, IBM WebSphere Application Server, Red Hat JBoss, Apache Tomcat, or .NET Framework/IIS.
- **Embedded service runtime** — The service runs in its own embedded runtime. Unlike an enterprise app server, which may be hosting multiple applications per instance, the embedded runtimes are one runtime per application. Examples include Spring Boot, Quarkus, Python with Flask/FastAPI, Node.js Express web framework.
- **Packaged application or commercial off-the-shelf (COTS) deployment** — The service exposes capabilities of a packaged or COTS application that are typically implemented as a monolith.
- **Serverless function platform as a service (FaaS)** — The service is implemented as a cloud-native serverless function (such as Azure Functions or AWS Lambda) that can be invoked as an API.

This research note is restricted to the personal use of .

- **An existing API that will be published to new consumers —** In this implementation you are not responsible for the service implementation. You just want to publish an existing API to new customers and need to implement additional/new mediation logic. Examples include third-party APIs to be consumed by internal consumers and APIs originally intended for a single application consumption scope that are now being published to broader consumption within the organization or to external consumers.

Step 2: What Is the Scope of the API Exposed by Your Service?

The intended use case for your API is an important decision factor when you are publishing an API to new consumers. Use cases also dictate the kinds of developer support (automation or self-service), monitoring, security/identity, and other capabilities you need from your API mediator.

The choices for use case are:

- **A — Enterprise scope:** An API with enterprise scope is designed for sharing and reuse by a broad-based audience, including intended and potentially unanticipated use cases throughout and outside the enterprise. An API shared within a business unit or geographic division would also fall into this category.
  - Furthermore, the API must be secured in accordance with enterprise information security and network security policies. Examples of enterprise-scoped APIs include published APIs that are shared for reuse externally to business partners or customers, internally across an enterprise (enterprise-scoped) or within an organizational unit or division.
  - Enterprise-scoped APIs should be published to a developer portal or catalog to support design-time discovery by API users and sharing of documentation, API specifications, and operational information.

- **B — Application scope/subapplication scope:** An application-scoped or subapplication-scoped API is designed for a specific use case. The API exposes application-specific functionality and is often not appropriate for reuse outside the application/project for which it is designed.
  - Application-scoped APIs and their consuming components are typically designed and developed by the same project or product team responsible for managing their life cycle.

This research note is restricted to the personal use of .

Application-scoped APIs are often not published to a developer portal for cross-domain consumption because of their limited applicability outside of the immediate development team. Despite being unpublished, application-scoped APIs should still be registered with an API catalog, managed and secured. Some modern developer portals allow application-scoped APIs to be published to a limited audience (such as the development team) using fine-grained access control. Alternatively, internal developer portals, such as Backstage, Cortex, Port and OpsLevel, are an increasingly popular option for cataloging APIs, services and other software assets. See How to Implement Internal Developer Portals for details.

All APIs, regardless of enterprise or application scope, should be discovered, cataloged and managed. If the APIs are shared internally or externally, they should be published to the appropriate developer portal. Organizations should use multiple portals to support the needs of different types of consumers. (See Quick Answer: Use Separate API Portals for Internal and External Developers for detail.)

When the API is enterprise-scoped, proceed to **Step 3** to evaluate the suitability of an enterprise API gateway.

Proceed to **Step 4** to evaluate a lightweight API gateway if the API is used at an application scope or even subapplication/component scope.

Step 3: Does the API Meet the Enterprise API Gateway Criteria?

Your answers to prior questions in the decision flow suggest that an enterprise API gateway is a good candidate to mediate your API. This is because:

- The APIs your service implements are enterprise-scoped and designed for sharing and reuse as an enterprise asset.
- You arrive here via a “fail-up” path after considering the criteria for a lightweight gateway.

Review some additional criteria to validate this choice. The key criteria include:

- **Protected at the edge/perimeter** — API protection and API access control at the edge of the enterprise are essential to your overall API security strategy.
- **Exposed outside of the enterprise** — The API is published to consumers outside the enterprise as an external, private or partner API.

This research note is restricted to the personal use of .

- **Widely shared within the enterprise** — The API is widely shared and consumed by internal applications or services.
- **Managed as a product** — The API is productized and published in a developer portal or API marketplace.
- **Business-value-driven** — The API is aimed to contribute direct and indirect revenue to the business.

The criteria are described in detail in the Criteria for Using an Enterprise API Gateway section of this research.

If you answer "yes" to one or more of the criteria, you should select an enterprise API gateway as the mediation technology for your service's outer API.

If you answer "no" to all listed criteria for an enterprise API gateway, you should consider a lightweight API gateway by proceeding to **Step 4** of the decision flow. Many lightweight gateways offer the same capabilities as enterprise gateways.

In the unlikely case that you find that none of the criteria for an enterprise API gateway or a lightweight gateway meets your needs, you should request an inquiry with a Gartner analyst to discuss your unique situation, requirements or constraints. Keep in mind that all services used for production applications should be mediated and managed.

Step 4: Does the API Meet the Lightweight API Gateway Criteria?

So far, your answers indicate that a lightweight API gateway is the best option for your scenario, but you'll need to review additional criteria to validate this choice.

The key criteria include:

- **APIs published within an application domain** — You have APIs that are consumed with the context to an application or a group of related applications by a product team. These may be backend for frontend services or private APIs used exclusively in an application.
- **API traffic kept inside your network/domain perimeter** — You're required to confine your internal API traffic within your trusted network for security and compliance reasons.

This research note is restricted to the personal use of .

- **Application-specific, fine-grained policies** — Your application needs to impose certain domain-specific, fine-grained API policies that are only applicable within the microperimeter of your application.
- **Developer-centric DevOps automation** — You have a requirement/desire to deploy API mediation policy as part of your DevOps process.

These criteria are described in detail in the Criteria for Using a Lightweight API Gateway section of this research.

If you answer "yes" to one or more of the criteria, then you should select a lightweight API gateway as the mediation technology for your service's API.

However, if you answered "no" to all listed criteria for a lightweight API gateway, proceed to **Step 3** and consider an enterprise API gateway because it may offer the features you need.

In the unlikely case that you find that none of the criteria for an enterprise API gateway or a lightweight gateway meets your needs, you should request an inquiry with a Gartner analyst to discuss your unique situation, requirements or constraints. Keep in mind that all services used for production applications should be mediated and managed.

Step 5: Does Your Service Participate in Intradomain or Cross-Domain Communication?

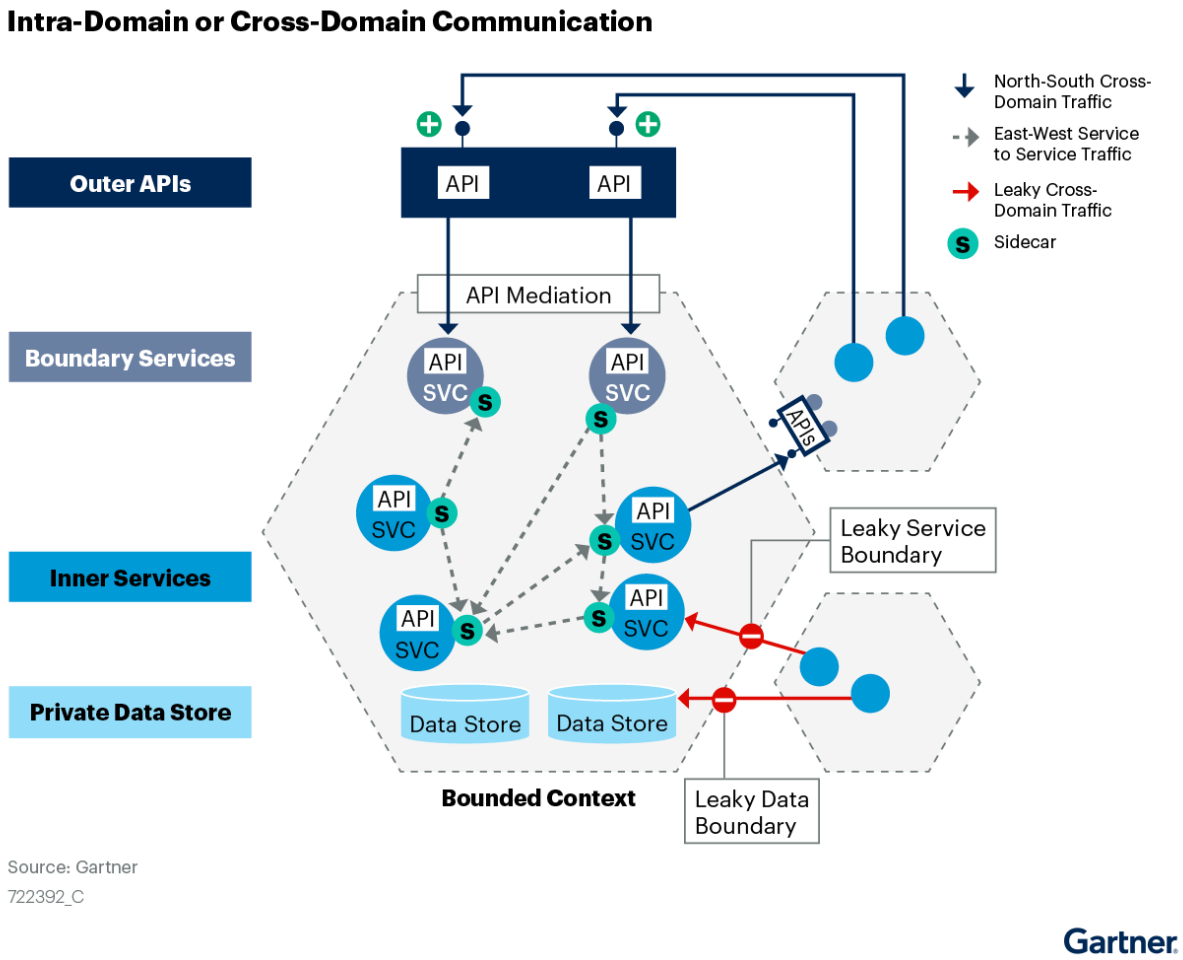
You're at the step because you have indicated one of the following in Step 1:

- Your service is running in a managed container system.
- Your service is running in a VM, and you have access to a service mesh that supports VM workload.

This question expands on whether the service-to-service interaction is intradomain or cross-domain, as depicted in Figure 3.

This research note is restricted to the personal use of .

Figure 3: Intradomain or Cross-Domain Communication



A domain defines a logical boundary around a group of interrelated services that implement related business functions, using a common logical data model, business language and vocabulary. These services adopt the same vocabulary to describe the behaviors of their collective business capabilities. Because of the close-knit nature of these miniservices or microservices, they can be considered part of a bounded context representing a business domain or subdomain, as shown in Figure 3. Services in a bounded context are highly cohesive and share a common goal or purpose; however, they take on different roles when they interact with other services inside and outside their domains.

The majority of the services are “insiders;” they only communicate with services inside the domain, using their inner APIs. Such intradomain communication is also referred to as east-west traffic.

This research note is restricted to the personal use of .

A few services are designated as boundary services; they represent and advertise the domain's core behaviors or capabilities accessed by outside systems. Boundary services encapsulate the domain capabilities and facilitate interdomain communication. All of the domain's services need to be secured, monitored and managed.

There should be no direct access into the domain other than via the gateway (or other managed ingress point). The API gateway is given privileged access to proxy-authorized requests to boundary services to map their inner APIs to the outer API endpoints exposed by the gateway. Requests that come from outside the domain through the API mediation layer are called north-south traffic.

Note: Boundary services can interact with inner services via east-west traffic on the back side.

If the service participates in cross-domain, north-south communication, then proceed to **Step 6** to consider an ingress gateway to mediate the service's inner API to its outer API.

If the service engages in intradomain, east-west traffic, then go to **Step 7** to consider a service mesh to mediate service-to-service communication.

Some services may participate in both cross-domain and intradomain communication; in that case, visit **Step 6** and **Step 7**.

Step 6: Does the API Meet the Ingress Gateway Criteria?

Your answers to prior questions in the decision flow suggest that you should consider an ingress gateway to mediate your service's API This is due to one of the following:

- Your service is a boundary service, engaging in north-south communication.
- Your service runs in a container orchestration platform.
- Your service runs in a service mesh (prior to entering this decision flow).

Review these additional criteria to validate this choice. The key criteria include:

- **Secured access to container management cluster or service mesh** — You must control external access to services running in Kubernetes or in a service mesh. You may also desire to stay within the Kubernetes API set (such as Ingress Controller or Kubernetes Gateway API) for your implementation.

This research note is restricted to the personal use of .

- **Multitenant delegated ingress control** —You want to delegate ingress routing rules to individual development teams, so they can share compute resources in a container management platform or service mesh while maintaining some autonomy with their ingress configuration.
- **Application-centric traffic management** — Application teams need fine-grained control to manage inbound traffic to their application components and services.
- **Declarative, desired-state configuration** — You may prefer a declarative configuration model to manage API mediation and service deployment.

The criteria are described in detail in the **Criteria for Using an Ingress Gateway** section of this research.

If you answer "yes" to one or more of the criteria, then you should select an ingress gateway as the mediation technology for your API.

However, if you answered "no" to all listed criteria for an ingress gateway, you should evaluate a lightweight API gateway described in **Step 4** of the decision flow.

Step 7: Does the Service Meet the Service Mesh Criteria?

Your answers to prior questions in the decision flow suggest that you should consider a service mesh to manage and facilitate service-to-service communication. You may validate your service's suitability for service mesh in the Criteria for Using a Service Mesh section of this research, as well as in When to Use a Service Mesh in Cloud-Native Architectures for in depth discussion.

The key criteria include:

- **Dynamic environment** — You have a dynamic, frequently changing microservices environment with variable workloads that requires service to scale up and down.
- **Number of service interconnects** — You have a sufficient number of service-to-service interconnects so the benefits of a service mesh outweigh the overhead of operating it.
- **Zero-trust security using mutual Transport Security Layer (mTLS)** — You need to authenticate and encrypt traffic between services using mTLS.

This research note is restricted to the personal use of .

- **Request authentication and authorization using JSON web tokens (JWT)** — You need to authenticate/authorize the caller using JWT before granting access to the service.
- **Observability and dependency analysis** — Observability to service behaviors, topology and service dependency is important to your application and infrastructure and operations (I&O) teams.
- **Service traffic management** — Fine-grained traffic management capabilities are critical to your microservices architecture.

If you answer "yes" to one or more of the criteria, then you should select service mesh as the mediation technology for your service-to-service communication.

However, if you answered "no" to all listed criteria, your application will not gain many benefits from a service mesh because of one of the following reasons:

- The services don't generate much east-west traffic.
- The overhead of running a service mesh exceeds the resource required for the application itself.
- Your application retains a mostly static network topology without the need to adjust for a dynamic routing policy or to support frequent component updates.

The suitability of a service mesh is based on the collective benefit from a group of services, rather than individual ones. You may not have a sufficient number of services to meet the minimum criteria for a service mesh today. In this case, you will need to rely on the built-in DNS, service discovery, and the container orchestration platform's monitoring and routing features, such as Kubernetes.

However, as you add more services to your application or as new requirements arise, you should reevaluate the suitability of a service mesh again by revisiting the decision flow.

Principles, Requirements and Constraints

The decision flow briefly illustrates the logic of how to arrive at an effective mediation technology for a specific use case. This section reveals requirements and constraints that form the underlying logic of the decision flow.

This research note is restricted to the personal use of .

Requirements and Constraints

The requirements and constraints described in this research represent the typical considerations that will influence your meditation technology selection. Not all criteria apply to every decision, and you may have situations where you'll need to consider additional criteria not included in this section. As you explore your requirements, keep in mind that every decision involves a trade-off of conflicting priorities.

Criteria applicable to many frequently observed integration/mediation use cases include:

- Integrating with existing infrastructure
- Leveraging integration technologies
- Combining synchronous and asynchronous communication
- Optimizing your enterprise governance model for APIs and services
- Collaborating with platform team

Criteria specific to a particular mediation technology include:

- Criteria for using an enterprise API gateway
- Criteria for using a lightweight API gateway
- Criteria for using an ingress gateway
- Criteria for using a service mesh

Integrating With Existing Infrastructure

Most organizations have a combination of existing application delivery controllers (ADCs), web application and API protection (WAAP), API gateways, container management platforms, enterprise service buses and integration platform as a service (iPaaS) in their on-premises and cloud environments. You may leverage your existing enterprise API gateway and application and integration infrastructure for API mediation if they align well with your requirements.

This research note is restricted to the personal use of .

For example, you can use an existing enterprise gateway for application-scoped APIs if your DevOps team (or teams) is not ready to operate lightweight gateways dedicated to a particular application. You may tactically consider an API gateway supplied by your iPaaS solution if you don't have an enterprise API gateway.

If your APIs are deployed in hybrid or multicloud, then the proximity between your API consumers and API providers can help determine the placement of your individual API gateway instances and the overall APIM deployment topology. (For guidance on optimizing APIM for hybrid and multicloud, see Comparing Architectures for Hybrid and Multicloud API Management.)

Your service runtime infrastructures — VMs, serverless FaaS and container management platforms — might also influence or constrain your choices of API gateway form factors (physical appliances, VM, container or embedded, etc.). For example, services deployed in an fPaaS (e.g., AWS Lambda function) may require that the native API gateway (e.g., Amazon API Gateway) be from the same cloud provider.

If you need to support service-to-service communication across heterogeneous runtime infrastructures (e.g., VMs and containerized services), then service meshes with such capability are relatively nascent. Use this functionality with caution, and only use it if you have sufficient requirements to justify its adoption.

Leveraging Integration Technologies

Mediation technologies are designed to provide governance capabilities — such as access control, traffic management and protocol mediation — that do not include business logic. Table 1 describes the ideal, possible or inappropriate workloads for API mediation.

This research note is restricted to the personal use of .

Table 1: Workloads for API Mediation

Ideal	Possible	Inappropriate
▪ Policy enforcement ▪ Usage monitoring ▪ Traffic management	▪ Protocol mediation (e.g., converting SOAP to REST) ▪ Format translation (e.g., converting XML to JSON) ▪ Simple payload modification (e.g., filtering sensitive fields out of objects) ▪ Simple orchestration	▪ Transformation that involves business logic ▪ Content enrichment from another service ▪ Complex orchestration (e.g., invoking multiple inner APIs to construct a response to a single inbound call) ▪ Complex payload modification

Source: Gartner

Adding complex mediation, transformation or orchestration logic in an API mediation gateway is an antipattern that leads to bloat and tight coupling and blurs the line between API policy enforcement and service implementation.

Functionality that involves business logic to implement complex data transformation, aggregation or service orchestration should be implemented in a separate service, rather than in the mediation layer. This logic could be implemented using code or using integration technologies, such as event stream processing, iPaaS, EDA, fPaaS, distributed integration platforms or integration frameworks.

Selecting the right combination of technologies for application integration and service implementation is outside the scope of this research; however, they are covered in Choosing Application Integration Platform Technology and Decision Point for API and Service Implementation Architecture, respectively.

This research note is restricted to the personal use of .

Combining Synchronous and Asynchronous Communication

Not all service-to-service communication uses synchronous request/response. An event-driven communication model can reduce coupling and dependency among services, thereby improving adaptability and flexibility. Common design patterns (e.g., event sourcing and command query responsibility segregation) are built on a synchronous API front end with an EDA. This uses an event broker to facilitate the flow of events between its command and query modules. Application technical professionals should combine request- or response-based mediation with an asynchronous event-driven model for modern application delivery.

Services that take advantage of EDA or asynchronous messaging connect directly to message (or event) brokers using broker-specific protocols or standards-based messaging protocols. Current API mediation technologies such as API gateway and service mesh typically provide limited support for EDA and asynchronous messaging and, therefore, may not be sufficient in mediating communication between the services and message (or event) brokers.

For additional guidance on EDA, review:

- Reference Architecture Brief: Event-Driven Application Architecture
- Choosing Event Brokers to Support Event-Driven Architecture
- Essential Patterns for Data-, Event- and Application-Centric Integration and Composition

Optimizing Your Enterprise Governance Model for APIs and Services

The enterprise governance model for API and services spans a spectrum from centralized control to federated autonomy. Organizations generally adopt a hybrid model somewhere along the spectrum from a centralized governance model to a federated governance model.

A centralized governance model emphasizes consistency and control over the API life cycle throughout the enterprise. Organizations typically implement this model around a centralized APIM team. For example, an API center of excellence might be responsible for sharing API best practices and ensuring consistency in API design and policy enforcement, using enterprise gateways and developer portals.

This research note is restricted to the personal use of .

A federated governance model delegates API governance to a number of decentralized line of business (LOB) teams or geographical locations. This model favors autonomy, agility and independence. Organizations adopting this model generally have diverse, autonomous LOBs that are empowered to optimize their governance model to their local environment or product needs.

If left unchecked, organizations with an overly decentralized governance model may face difficulties in maintaining consistency and visibility over their APIs and services. Under this model, product teams, along with I&O teams, are empowered to provision and operate lightweight API gateways, ingress gateways or service meshes for their application delivery.

A hybrid centralized and federated governance model enables organizations to achieve a balance between centralized consistency and control with decentralized autonomy and agility in distributed teams. Organizations would need to customize a suitable governance approach that aligns with their organizational structure and culture. A combination of enterprise API gateways (for global policies), a lightweight gateway and ingress gateway (for team autonomy and customized policies), and a service mesh (for consistent service-to-service communication) offers a full set of controls. Organizations can execute a hybrid governance model for APIs and services.

Your governance model frames how your organization implements and manages mediation technologies, such as APIM and service meshes. The governance model also determines the personas responsible for the operation, configuration and utilization of each mediation layer. The level of control and autonomy delegated to your development and product teams can enable or constrain their ability to effectively leverage mediation technologies for application development and delivery.

The mediated API architecture is typically hierarchical with multiple mediation layers forming a topology that resembles a fractal landscape. The mediation pattern repeats itself at every layer, albeit at different scopes of control and for different client requirements. For example, the outer APIs of a lightweight gateway could become the inner APIs of an enterprise gateway to tailor application-scoped APIs for external client consumption. Organizations should leverage an API gateway, an ingress gateway and a service mesh comprehensively to achieve effective governance throughout the enterprise.

This research note is restricted to the personal use of .

Collaborating With Platform Engineering

Platform engineering is the team that engineers, delivers, maintains and improves a self-service application platform or PaaS — including the CI/CD tool chain — for multiple agile application teams building custom software. The platform engineering team is responsible for the ongoing operations of API gateways, API and service catalogs, container management platforms, and service meshes. It typically provisions the control planes of these platforms and sets global policies, "sane" defaults and guardrails for the production team/DevOps team.

Some of the policies and configurations for particular applications, services and APIs are delegated to the responsible DevOps team to optimize development team autonomy and efficiency without compromising cross-team consistency and governance. Platform engineering teams commonly own the implementation and delivery of internal developer portals, which critically provide effective developer experiences. See How to Implement Internal Developer Portals for more detail.

Refer to Use Platform Engineering to Scale and Accelerate DevOps Adoption for roles and responsibilities of platform operations. For guidance on managing operations of Kubernetes, see How to Prepare for Containers and Kubernetes and Designing and Operating DevOps Workflows to Deploy Containerized Applications With Kubernetes. For a discussion on service mesh platform operations, see Assessing Service Mesh for Use in Microservices Architectures.

Criteria for Using an Enterprise API Gateway

In the past, an enterprise API gateway was the most common API mediation technology deployed. It's traditionally used for governing the publication of APIs internally or externally to support integration, development and ecosystems as part of APIM.

Use the criteria checklist below to verify your decision to use an enterprise API gateway. If you answer "yes" to one or more of the criteria, then an enterprise API gateway is the appropriate mediation technology for your API. If an enterprise API gateway is not available, then you can consider a lightweight API gateway.

An enterprise gateway is effective when your APIs meet the following criteria.

Protected at the Edge/Perimeter

This research note is restricted to the personal use of .

Is API protection and API access control as the first line of defense and control at the edge essential to your overall API security strategy? Perimeter protection is an essential and necessary measure to secure external and partner-facing APIs at the edge while organizations adopt a distributed enforcement model to secure APIs across the enterprise.

An extended enterprise network may have multiple entry points. Traditional approaches to data center perimeter security — relying on firewalls, IP allow/block list, rate limiting and more — do not scale well to meet the challenge of a changing perimeter that encompasses public and private clouds. A more modern approach shifts focus toward identity and access management, exploit mitigation, bot mitigation, distributed denial of service and more. The modern approach targets common attack vectors using a combination of on-premises and cloud services at the edge.

API protection consists of API threat protection and access control, which are often achieved with a mix of technology, including a content delivery network, a web application firewall, ADC, WAAP and API gateways. Effective perimeter and edge protection requires consistent enforcement of global policies that are centrally managed and monitored. API security strategy requires a comprehensive methodology that addresses design, discovery, monitoring and protection. (For further insights, see Market Guide for Cloud Web Application and API Protection.)

Exposed Outside the Enterprise

Is this an external, private or partner API? Enterprise APIs that are exposed outside of the enterprise include your public/open APIs, APIs that support mobile and web apps, and private APIs dedicated to your partners and customers. In all cases, the consumers are outside the boundary of your enterprise.

Widely Shared Inside the Enterprise

Do you have APIs that are widely shared and consumed by internal applications and systems? The logical boundaries of enterprise networks extend to hybrid and multicloud via cloud connectivities, such as AWS Direct Connect, Google Cloud Dedicated Interconnect or Microsoft Azure ExpressRoute.

Internal API traffic may originate from an Amazon Virtual Private Cloud and traverse through AWS Direct Connect before reaching an API hosted in an on-premises data center. An enterprise API gateway can be used to secure and govern internal APIs.

Managed as a Product

This research note is restricted to the personal use of .

Is this API to be productized and published in a developer portal or API marketplace? The APIs are packaged and delivered to the customers as a product or as part of a product. This means that you are committed to stand behind the API product with a product manager and a delivery plan. This should include a developer portal or marketplace, full life cycle management, technical support and a product roadmap that aligns with customer demands.

Business-Value-Driven

Is the API aimed to contribute direct and indirect revenue to the business? Direct monetization of APIs generates revenue from charging for the consumption of APIs as products. For example, common API pricing models may include pay per call, revenue sharing, subscription or partnership. Indirect monetization of APIs delivers business values without a direct trace to revenue. Organizations may use the API to create new business channels to interact with customers, engage partners or increase brand exposure.

Criteria for Using a Lightweight API Gateway

Application architects sometimes ask, “Do I need a lightweight API gateway, if there is already an enterprise API gateway running at the edge of my enterprise network?” The answer is: “Yes, you do.”

If you’re using the API, then you need an API gateway between the API and its callers. If you don’t want internal users to route through your enterprise API gateway, then you also want a lightweight gateway for those use cases. More criteria questions like this help you validate the benefits of a lightweight API gateway.

If you answer “yes” to one or more of the criteria questions below, a lightweight API gateway is the appropriate mediation technology for your API.

APIs Published Within an Application Domain

Is this an internal API to an application or a group of related applications by a product team? Is this a private backend for frontend API for a particular application? Internal APIs may encapsulate domain capabilities to support cross-domain interactions within an application’s context. Immediate consumers of these APIs are within the boundary of the enterprise’s internal network. They may be private backend for frontend APIs designed to support specific user experiences or information delivery channels.

This research note is restricted to the personal use of .

These APIs are consumed exclusively by apps and devices outside of the enterprise network boundary, including web, single-page applications, mobile, voice and wearables. Even narrowly scoped APIs require strong protection in line with the API security strategy. Modern application architectures (such as MASA) enable multiple user experiences by leveraging multigrained services and application-scoped APIs. (See Create Agile Application Architecture With Composable Apps, APIs and Services for further insights.)

Keep API Traffic Inside Your Network/Domain Perimeter

Are you required to confine your internal API traffic within your trusted network for security and compliance reasons? Are you concerned about the latency introduced by routing API traffic through a centralized API gateway in the cloud or demilitarized zone when the API consumer and provider are colocated in the same trusted network? A strategically placed API gateway managed by or dedicated to a development team can optimize application network topology, reduce network latency, and improve API traffic isolation and segmentation within a designated perimeter.

Application-Specific, Fine-Grained Policies

Do you have application-specific, fine-grained policies for this API? Your application needs to impose certain domain-specific, fine-grained API policies that are only applicable within the microperimeter of your application. These may include:

- Authorization policies based on roles or attributes defined within the context of your application
- Traffic management/routing rules to support your blue-green or canary deployment
- Data redaction rules based on the user's persona or user roles

Developers of modern applications that use multigrained services are demanding a high degree of control over networking, traffic management, security and observability to optimize the application's runtime and operational aspects.

Developer-Centric DevOps Automation

This research note is restricted to the personal use of .

Do you have a requirement/desire to deploy API mediation policy as part of your DevOps process? Application-scoped APIs often manifest themselves as miniservices, exposing domain capabilities. They are independently deployable and are associated with a regular release cadence that aligns with the application release/update cycle. For this reason, application development teams prefer to manage their APIs' life cycle as an integral part of their application CI/CD process. Any external dependency on the enterprise API team to separately test and deploy APIs would impede the development team's agility, autonomy and control with their application CI/CD processes.

Criteria for Using an Ingress Gateway

An ingress gateway is the entry point to a container management platform, such as Kubernetes, or to a service mesh, such as AWS App Mesh. Application architects often ask, "Should I use an ingress gateway to secure and route external API traffic into a Kubernetes cluster or service mesh?"

If you answer "yes" to one or more of the criteria questions below, then you can take advantage of an ingress gateway for your services.

Secured Access to Container Management Cluster or Service Mesh

Do you have a requirement to control external access to the services running in Kubernetes or in a service mesh? An ingress gateway controls external access to applications and services hosted in a Kubernetes cluster or service mesh through well-defined entry points. It marks the demarcation point where north-south (outer) API traffic is handed off to east-west communication between services within a cluster or service mesh.

Multitenant Delegated Ingress Control

Do you have multiple applications running in the same container management cluster and sharing the same cluster ingress? Do you want to delegate ingress routing rules to individual development teams so they can manage uniform resource identifier paths assigned to their applications or services? Running multiple applications in the same Kubernetes cluster in a multitenant configuration presents some unique, delegated administration challenges at the point of ingress.

Certain ingress gateways, such as those that implement or plan to implement Kubernetes Gateway API (e.g., Contour and Solo.io Gloo Gateway), can share admin and routing controls between I&O and individual application development teams.

Application-Centric Traffic Management

This research note is restricted to the personal use of .

Is your application team empowered to manage application-level traffic flow to support CI/CD and release management best practices (e.g., blue-green or canary deployment)? Application teams need fine-grained control to manage inbound traffic to their application components and services.

Declarative, Desired-State Configuration

Do you prefer a declarative configuration model to manage API mediation and service deployment? Using declarative configurations expressed in a YAML or JSON file, developers can specify the desired states of the system without explicitly programming it in a sequence of instructions.

Criteria for Using a Service Mesh

Application architects often ask, “When should I use a service mesh?” or “Do my services meet the criteria for using a service mesh, as in Step 8 of the decision flow?”

If you answer “yes” to one or more of the criteria questions below, you should use a service mesh. See When to Use a Service Mesh in Cloud-Native Architectures for further guidance.

Dynamic Environment

Do you have a dynamic, frequently changing microservices environment with variable workloads that requires service to scale up and down? Microservices are ephemeral; existing services may scale up and down in response to changing demands. New services may come into existence to offer new capabilities or enhancements.

These new services can start interacting with existing services quickly and securely. As a result, microservices must be effectively discovered, tracked and monitored in a timely manner to ensure transactions are being fulfilled by the healthy service instances. Without automation, service management is complex and intractable for developers and I&O technical professionals.

Number of Service Interconnects

Does the number of service-to-service interconnects within your application domain exceed a threshold (perhaps five or more) so the benefits of a service mesh outweigh the overhead of operating it? A fair amount of east-west traffic and potentially dynamic service-to-service interconnect topology are good indicators for service mesh adoption.

Multicluster or Mixed Workload Service Connectivity

This research note is restricted to the personal use of .

Does your service-to-service connectivity span multiple clusters or across a containerized and VM workload? Do you have instances of a service deployed in an on-premises data center and the cloud where you want to optimize the transaction flow by calling the closest instance?

Zero-Trust Security Using mTLS

Do you need to encrypt traffic between services? Is mTLS required to access a service's API? To improve runtime governance on services, organizations adopt a zero-trust model to manage application security. Zero-trust principles take on the view of "never trust; always verify."

All interactions between services must be authenticated and authorized with microsegmentation down to the service level. Kubernetes and service mesh technologies can provide some segmentation/microsegmentation capability natively (e.g., using namespaces, role-based access controls and a label selector).

Many organizations shy away from implementing mTLS due to the complexity associated with server certificate issuance, signing and expiration. A service mesh can provide this capability to every service without explicit implementation by a developer because it provides its own certificate authority and can rotate certificates for services automatically. (For further guidance around network and infrastructure security within containerized environments, see Container Supply Chain: 10 Security Vulnerabilities and How to Address Them.)

Request Authentication and Authorization Using JWT

Is your service required to validate end-user credentials and/or entitlements before granting access? If so, a JWT attached to the request can help fulfill this requirement, especially when your service needs to enforce fine-grained authorization policies based on claims presented in the token.

A JWT is often used to hand off transaction context from north-south traffic to east-west traffic. A service mesh, such as Istio, has built-in capability in the sidecar to validate JWT in the request header and only forward authorized requests to the service.

Observability and Dependency Analysis

Is observability to service behaviors, topology and service dependency important to your application and I&O teams? Application developers need access to metrics in distributed systems for troubleshooting, maintaining and optimizing applications and their dynamic runtime environments.

This research note is restricted to the personal use of .

I&O professionals also demand that IT infrastructure monitoring tools provide visibility into microservices, containers and container orchestration platforms to optimize and coordinate resource utilization across hybrid infrastructures.

Service Traffic Management

Are fine-grained traffic management capabilities critical to your microservices architecture? These capabilities include rate limiting, circuit-breaker functionality, retries, load balancing, blue-green deployment and canary release.

Although these are also features of enterprise and lightweight API gateways, traffic management in a service mesh tends to be much more fine-grained and application-specific. Effective service traffic management is essential to a scalable and resilient microservices architecture, especially if you have a frequent release cadence and high rate of change.

Alternatives

This research helps you to choose between enterprise API gateway, lightweight API gateway, ingress gateway and service mesh. In addition to describing each of the alternatives for this research, the following sections will explain how these API mediation technologies work individually, and in combination, to meet common requirements and deployment patterns.

Enterprise API Gateway

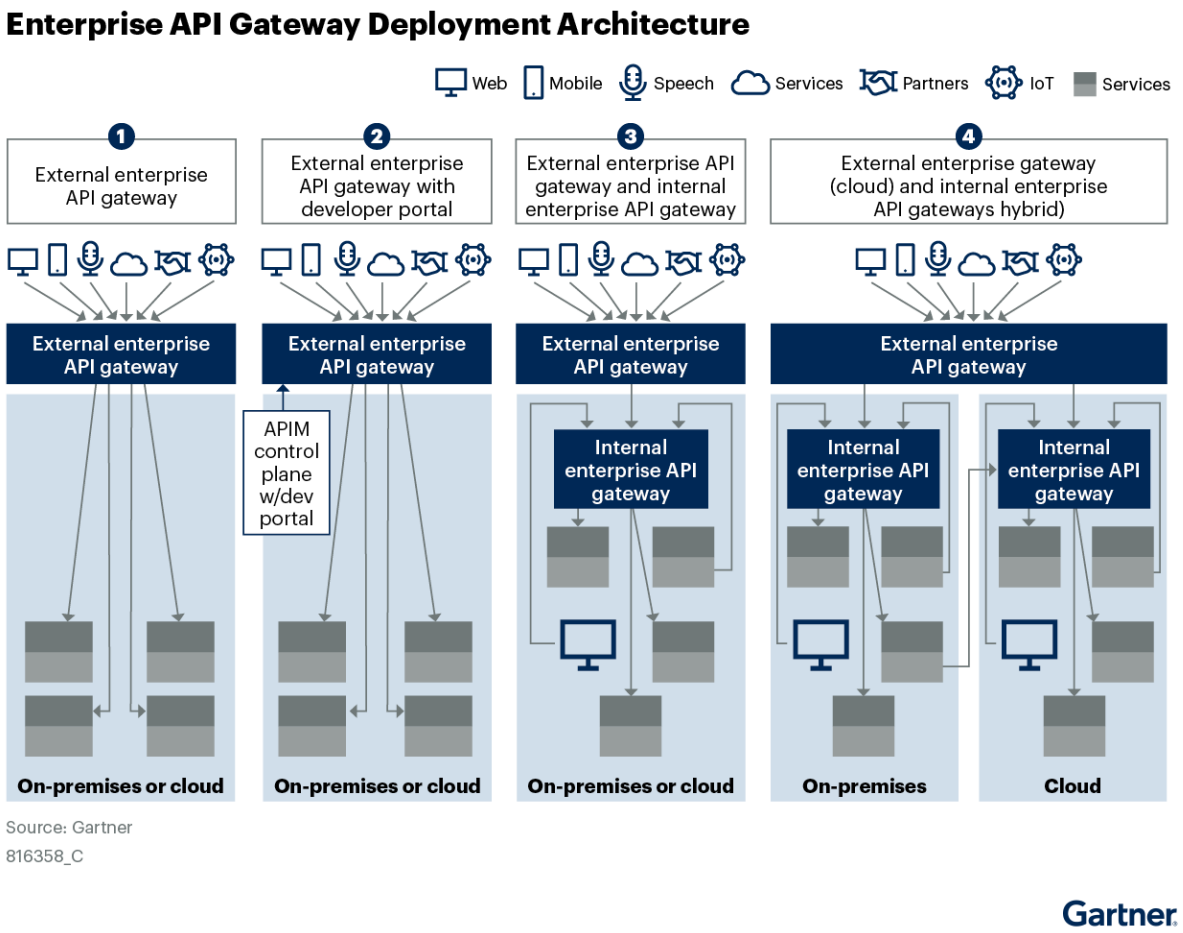
The evolution of API mediation began with the primary mission to securely expose business capabilities as APIs to external customers or partners. Enterprise API gateways fulfill this role by mediating all API traffic into and out of the enterprise network. Deployed at the edge of your enterprise network or security boundary, enterprise gateways use a centralized deployment model to provide the first line of defense at the entry point to your enterprise network or security zone. Being centralized does not mean enterprise gateway instances are singular. As enterprise networks expand from on-premises to hybrid and multicloud, multiple entry points to the enterprise will exist. Hence, enterprise gateways have many instances to secure and manage these entry points.

An enterprise API gateway is part of an APIM platform that includes an administration portal and a developer portal. API gateways are responsible for policy enforcement on the data plane. The admin and developer portals, along with the APIM control plane, address other aspects associated with managing APIs, such as design, versioning discovery, registration, consumption, monitoring and analytics.

This research note is restricted to the personal use of .

Organizations use enterprise API gateways in a variety of configurations to meet their API protection and management needs, often with (but sometimes without) a full life cycle APIM platform. Figure 4 illustrates a few representative examples of how enterprise API gateways are deployed in the enterprise to protect external and/or internal API traffic.

Figure 4: Enterprise API Gateway Deployment Architecture



Here are four examples of enterprise API gateway architectures:

- **External enterprise API gateway only** — Organizations may choose not to deploy a developer portal because a few APIs are published to a limited number of partners and private consumers via the enterprise API gateway.
- **External enterprise API gateway with developer portal** — An enterprise API gateway is deployed to secure a large number of external APIs that are productized and published as enterprise offerings. A developer portal is provided for API developers to discover, test-drive, subscribe to and analyze the use of APIs from multiple providers.

This research note is restricted to the personal use of .

- **External and internal enterprise API gateways —** APIs are published to internal and external consumers. The APIs require separate mediators — internal and external enterprise API gateways — to support all requirements. For example, externally published APIs require different security and data redaction policies from those internally consumed.

- **Provider-managed external API gateway and self-hosted hybrid cloud internal API gateway —** APIs are hosted in the cloud and on-premises data centers. Most APIs are published and shared internally within the extended enterprise network. Some APIs are published to developers and partners outside of the enterprise. In this case, externally consumed APIs are secured by SaaS enterprise API gateways to leverage cloud elasticity and availability. To confine internal API traffic within the enterprise, internal enterprise gateway instances are strategically placed in close proximity to APIs running on-premises and in the cloud. Inbound API requests through the external gateways are routed to internal gateways so aggregate rate limiting to back-end services can be accurately enforced by the internal gateway.

Lightweight API Gateway

As APIs become an essential element of application development, API gateways are evolving from centralized, one-size-fits-all, enterprise-level components to distributed, lightweight, specialized policy enforcement points for individual applications or service domains. Dedicated gateways per application or domain responsibility offer better isolation and fault tolerance against resource contention or application failure.

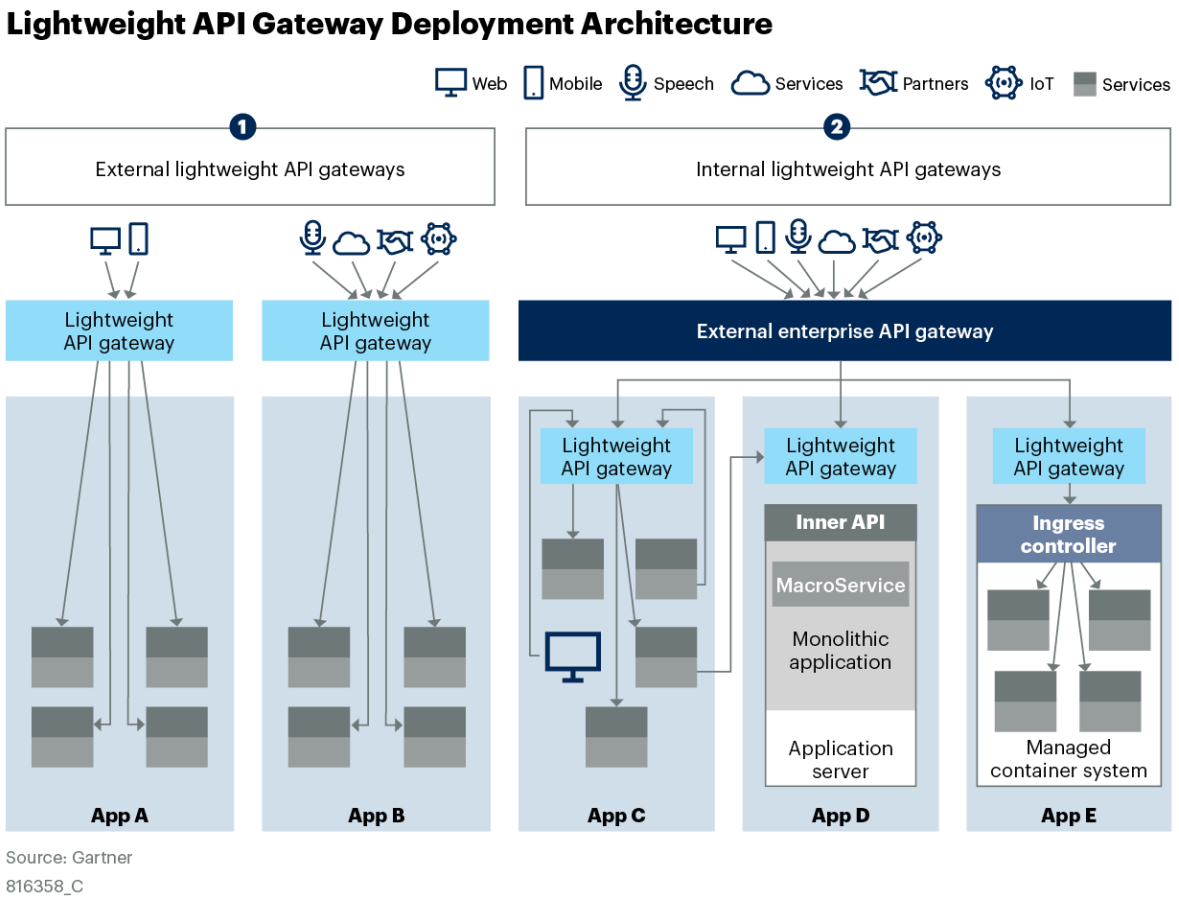
A lightweight gateway (sometimes known as a microgateway) is lighter in footprint compared with an enterprise gateway. It uses a distributed deployment model to provide last-mile defense at an API endpoint. A lightweight gateway is more dynamically configurable and focuses more on security, traffic management and observability and less on complex mediation logics.

Some vendors offer a large-footprint enterprise gateway and a separate lightweight gateway for distributed deployment. Other vendors implement a highly extensible API gateway that can be configured as an enterprise gateway or a lightweight gateway. Most lightweight gateways are containerized and deployable in container orchestration platforms, such as Kubernetes. Some API gateways or gateway components are even embeddable as part of the service runtime, application runtime or application platform such as Spring Cloud Gateway and Treblle.

Figure 5 shows sample use cases for lightweight API gateways.

This research note is restricted to the personal use of .

Figure 5: Lightweight API Gateway Deployment Architecture



Two common lightweight API gateway deployment architectures are:

- **External lightweight API gateways** — An external-facing lightweight API gateway is dedicated to a group of application-scoped APIs. They are consumed by their respective client applications. In pattern 1 of Figure 5, this is a web app and mobile app for App A and a mobile app, chatbot and Internet of Things devices for App B. An enterprise API gateway is not used because the APIs exposed by the respective lightweight gateway are not published and have no applicability outside the application. The lightweight gateways enforce all security policies required for externally facing APIs.

This research note is restricted to the personal use of .

- **Internal lightweight API gateways —** An internal-facing lightweight API gateway is dedicated to a group of application-scoped APIs. They are consumed primarily by components of the same application or sometimes by services from another application (as shown by the pattern 2 of Figure 5). Lightweight gateways can be used for services of different granularity, ranging from macroservices for monolithic apps (App D) to miniservices running in VMs (App C). Service mesh and/or ingress gateways are better mediation choices for containerized services running in container management platforms, such as Kubernetes (App E).

This pattern is topologically similar to Pattern 4 in Figure 4. The lightweight gateways here are fulfilling the role of the internal enterprise gateway but on a more granular scale. A lightweight gateway operates in application scope, whereas the internal enterprise gateway governs APIs in the entire enterprise application tier.

Lightweight API gateways come in varying ranges of capabilities, from the cloud-native API gateways (e.g., Traefik Hub, Solo.io Gloo Gateway, Ambassador Labs Emissary-Ingress and MuleSoft Flex Gateway) to embedded gateways (e.g., Spring Cloud Gateway). Feature-rich lightweight gateways, such as Kong Gateway, can also be deployed as enterprise API gateways.

Ingress Gateway

An ingress gateway refers to the entry point of a Kubernetes cluster or service mesh that bridges inbound (north-south) traffic from outside of the cluster to services within the cluster. Two variations of ingress gateways exist, depending on the container orchestration platforms and/or service mesh implementations:

- **Kubernetes ingress/gateway controller** is an ingress resource typically provisioned with a Layer 7 load balancer that routes external HTTP traffic into the cluster according to the ingress rules. Base ingress controller capabilities include load balancing, TLS termination and virtual hosting. Most common ingress/gateway controller implementations include NGINX, HAProxy and Envoy. Kubernetes' ingress/gateway controller is a stand-alone component that can work with or without a service mesh.

This research note is restricted to the personal use of .

- **A service mesh ingress gateway** is a built-in component of a service mesh that is responsible for bringing north-south traffic into the mesh. It improves on the base ingress controller by offering additional traffic management, security and observability capabilities that come with the service mesh. The most popular ingress gateway implementation today is Envoy, which is also used by many service meshes as sidecar or node car to facilitate east-west traffic among services. By standardizing on Envoy as the data plane, service mesh products can leverage the same control plane to manage both north-south and east-west traffic across the mesh.

For more details on the role of an ingress controller or ingress gateway, see Assessing Kubernetes for Hybrid and Multicloud Application Portability.

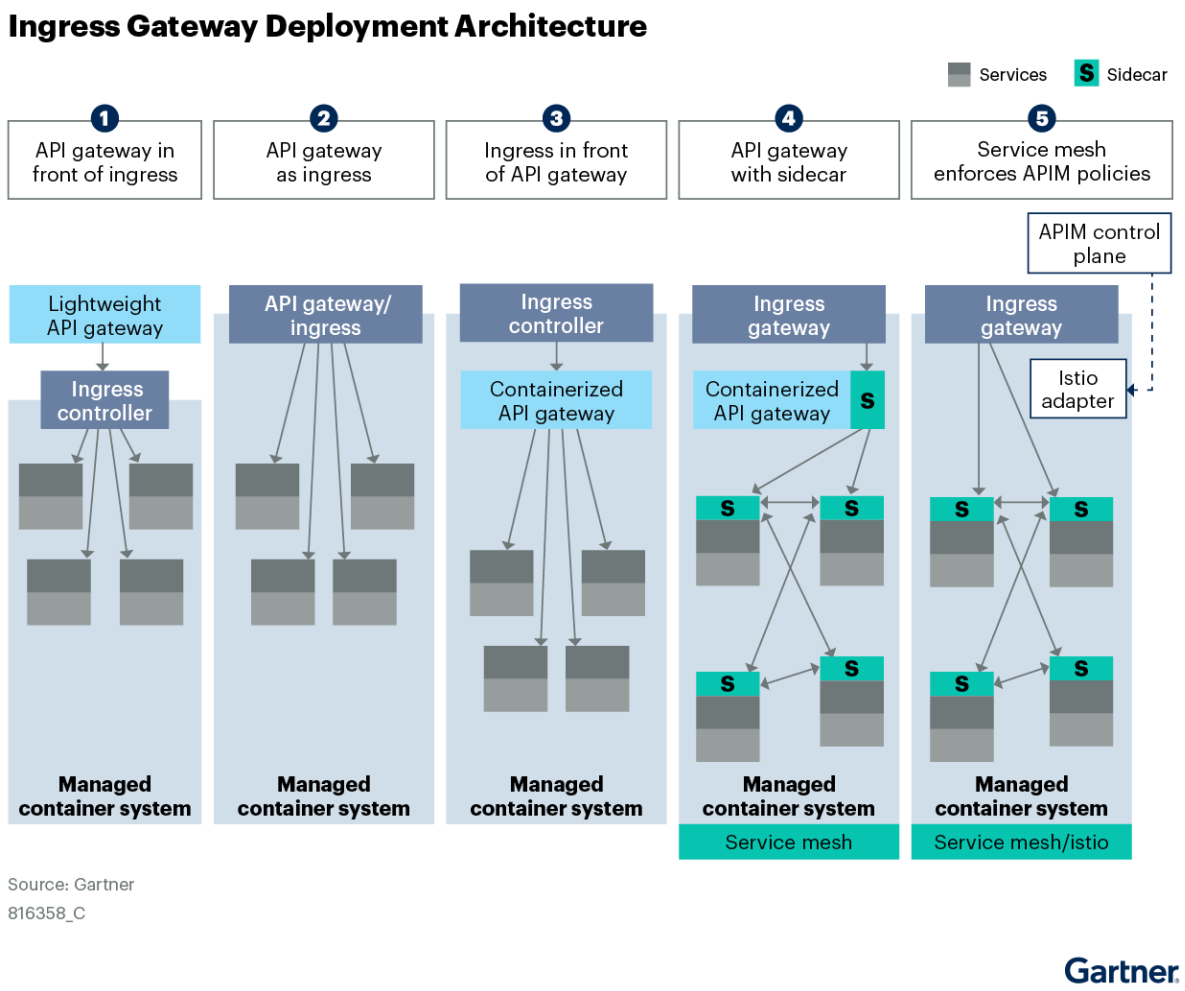
Once a service is exposed at the edge of a Kubernetes cluster or service mesh, it embodies the cluster's behavior by encapsulating its domain capabilities. The service needs to be mediated as an outer API for other systems, applications or components to call on. A standard ingress gateway or ingress controller can load balance and route API traffic. However, it is not an API gateway and does not provide basic API mediation capabilities, such as API life cycle management, API key validation and API monitoring.

The key distinction between an ingress gateway and an API gateway is context. An API gateway operates on the basis of an API, with an emphasis on ease of consumption. An ingress gateway focuses on service exposure and service traffic management, with an emphasis on operational efficiency, consistency and resilience.

Technical professionals can use five methods to integrate or combine API gateway and ingress gateway capabilities (see Figure 6).

This research note is restricted to the personal use of .

Figure 6: Ingress Gateway Deployment Architecture



These methods include:

- **API gateway in front of ingress** — An API gateway is deployed outside of the Kubernetes cluster, sending inbound API traffic through the ingress controller into the cluster. In this architecture, configurations of the Kubernetes ingress and the API gateway need to be coordinated and synchronized at service deployment time. This model works best with an existing API gateway that runs outside a container orchestration platform.

This research note is restricted to the personal use of .

- **API gateway as ingress** — An API gateway is deployed in the place of an ingress controller or gateway controller. The standard ingress configuration has rather basic routing features and should be used only for backward compatibility reasons. Configuring an API gateway to honor Kubernetes ingress configuration can effectively disable its more advanced security and traffic management features. Technical professionals who select this architecture should use the API gateway's native configurations (through a CRD or an operator) to fully leverage its API gateway capabilities. Sample API gateway as ingress include Ambassador Labs Emissary-Ingress and WSO2 APK.

- **Ingress controller in front of API gateways** — API gateways are deployed inside the Kubernetes cluster, receiving inbound API traffic from the standard ingress controller. The API gateway must be responsible for routing API traffic to target services within or outside the cluster. There is no restriction on the number of API gateway instances deployed behind the ingress controller. Each API gateway instance can be dedicated to a development team to manage APIs for a namespace or bounded context.

- **API gateway with sidecar** — An API gateway is deployed in the cluster as a pod with a sidecar proxy. The API gateway receives inbound traffic from the ingress gateway and communicates with target services within the mesh via its sidecar.

- **Service mesh enforces APIM policy** — The APIM control plane integrates with a service mesh, such as Istio, by pushing its policies to an adapter. The adapter, in turn, configures the ingress gateway and/or service mesh control plane to enforce the APIM policies. Solo.io Gloo Mesh Gateway is an example of an API gateway that integrates natively with a service mesh, such as Istio.

Service Mesh

A service mesh uses a distributed model to manage service-to-service communications (east-west traffic) for miniservices and microservices deployed in clustered container systems.

A service mesh uses a distributed model that abstracts out cross-cutting concerns with clean separation of duty and responsibility. It logically separates its functionality into a control plane and data plane:

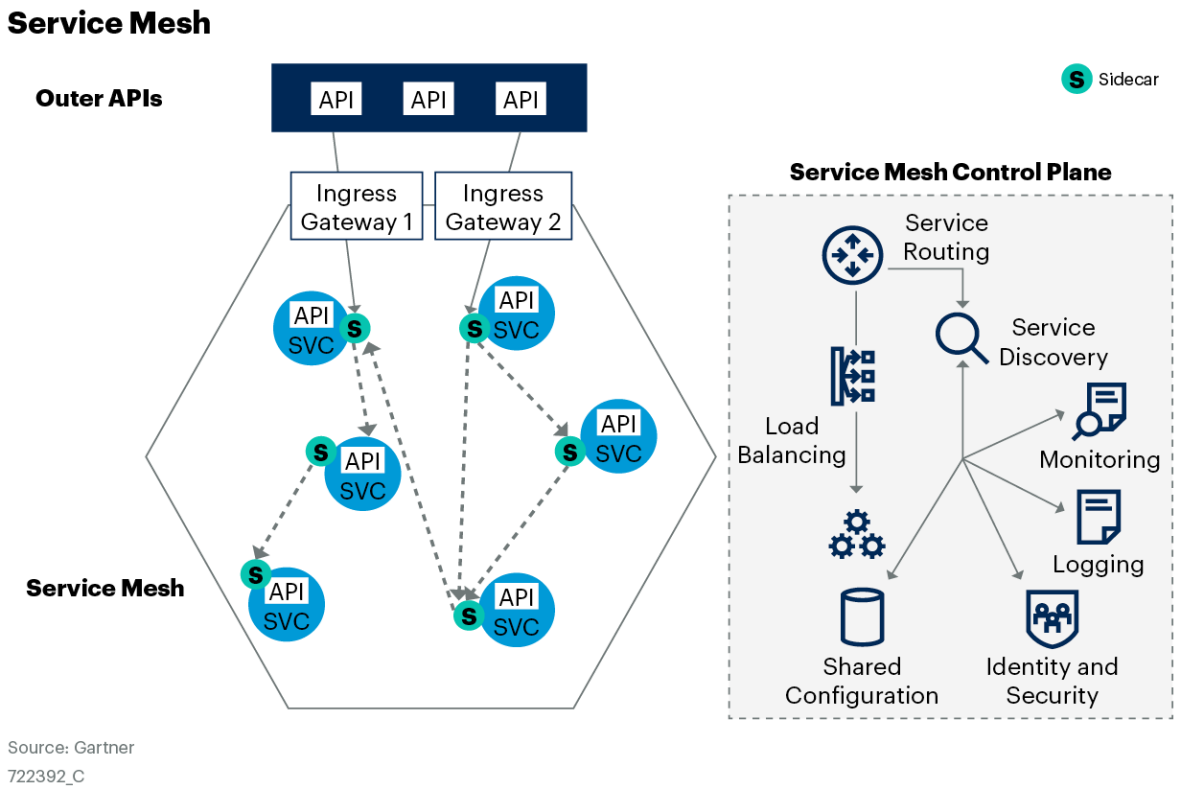
- The **data plane** is a collection of software-defined distributed agents, often known as sidecars, that manage service-to-service communication for miniservices and microservices.

This research note is restricted to the personal use of .

- The **control plane** acts as the coordinator and facilitator for the whole system. It keeps track of system states, manages configurations and coordinates behaviors of the data plane. The control plane does so by offering a set of housekeeping functionality, including service discovery and registration, monitoring and observability, traffic management, and security policy management.

A service mesh data plane can take on different forms. The most common ones are sidecar proxy, node proxy, stand-alone proxy or embedded library. Figure 7 depicts a service mesh implementation using sidecar proxies and a control plane.

Figure 7: Service Mesh



Source: Gartner
722392_C

This research note is restricted to the personal use of .

The sidecar proxy is deployed side-by-side with the individual services. The sidecar intercepts all network communications in and out of the service and manages them according to security and traffic management policies given by the control plane. The sidecars communicate with each other on behalf of the services. When mTLS is required, a sidecar acquires the necessary certificate from the control plane and connects securely with the target service. Services interact with each other and the outside world via their respective sidecar proxy. When traffic is congested or in the event of service failure, the sidecars also manage load balancing, retry or circuit breaking.

The service mesh model is well-suited to manage the dynamic and ephemeral nature of microservices and their complex interactions and interdependencies. (For further insights, see When to Use a Service Mesh in Cloud-Native Architectures.)

Future Developments

APIM products extend the reach of the APIM control plane from the edge of the enterprise to the edge and the interior of the service mesh.

Convergence of API Governance and Service Governance

APIM products extend the reach of the APIM control plane from the enterprise's edge to the service mesh's edge and interior. Some APIM vendors are offering API gateway runtimes that can be configured as an ingress gateway to a Kubernetes cluster or service mesh. A few APIM vendors, such as Kong, went further to offer their own implementations of a service mesh to manage microservices.

Other API vendors — such as MuleSoft, IBM API Connect and Google Apigee — focus on sharing governance policies between API gateways and service mesh through control plane interoperability, such as service discovery, network management and access management.

Service mesh vendors are taking an inside-out approach, evolving from a service-centric governance model to APIM at the edge of the service mesh. They are infusing APIM capabilities to the ingress gateway, while adding support for developer portals that can automatically discover and catalog services as candidates for API mediation. Gartner is seeing a rise in lightweight API gateway offerings, particularly those based on Envoy Proxy (e.g., Ambassador Labs Emissary-Ingress, MuleSoft Flex Gateway, Solo.io Gloo Gateway and Envoy Gateway).

This research note is restricted to the personal use of .

Another notable development at the interaction of service mesh and API gateway is the Kubernetes Gateway API specification, as the successor to the ingress and service API. [1] Kubernetes Gateway API (general availability as of December 2023). expands the monolithic management model of ingress controller by creating delegated role-based separation of duties among the platform operator, NetOps and application development teams.

Many API gateway and service mesh vendors are implementing support for Kubernetes Gateway API in their products. [2] We expect Kubernetes Gateway API to consolidate and unify the configuration and management of Kubernetes and service mesh ingress in the near future.

If this trend continues, we expect to see a convergence of API governance and service governance, where control plane interoperability plays a key role in enabling the exchange of policies and extending the reach of operational and transactional observability. We are also predicting that service mesh will become an integral capability of container management platforms (e.g., Google Anthos and VMware Tanzu) or will encompass multiple runtime infrastructures, including heterogeneous container management clusters and VMs (e.g., AWS App Mesh and Kong Mesh).

Innovation of the Service Mesh Control Plane and Meta Control Plane

The growing popularity of service meshes has fueled numerous developments and innovations in service mesh offerings. The majority of such development has focused on the control plane or meta control plane for service meshes, while the technology choice for the data plane (proxy) has largely settled on Envoy.

Notable recent developments include:

- **Hybrid infrastructure service mesh** — This service mesh can run on many different infrastructures, including VM, containers, serverless bare-metal hardware and so on (e.g., Consul, Istio, Kong Mesh, and AWS App Mesh).
- **Multicluster service mesh** — This service mesh spans multiple (Kubernetes) clusters, with secured communication among the clusters. This is accomplished by using a shared control plane for all clusters or a replicated control plane in each cluster.

This research note is restricted to the personal use of .

- **Multimesh federation** — These multiple independent service meshes can run workloads collaboratively within or cross-organizational boundaries. Interoperability is achieved by standardizing on a set of APIs that enable exchange of service discovery information, DNS and certificate management. Sample vendor products include Solo.io Gloo Mesh, Tetrate Service Bridge and OpenShift Service Mesh Federation.

- **Gateway API for Mesh Management and Administration (GAMMA)** — GAMMA is a workstream within the Kubernetes Gateway API project that tries to define service mesh configuration within a cluster. With Gateway APIs for Ingress handling north-south traffic into the cluster, GAMMA aims to pick up ingress traffic and transition them to service-to-service communication (east-west traffic) under the same unified Gateway API specification. This initiative effectively replaces the now archived Service Mesh Interface project in the Cloud Native Computing Foundation.

Evidence

[1] Kubernetes Gateway API, Gateway API.

[2] Kubernetes Gateway API Implementations, Gateway API

Document Revision History

Decision Point for Mediating API and Microservices Communication - 18 October 2022

Decision Point for Mediating API and Microservices Communication - 13 August 2020

Recommended by the Author

Some documents may not be available as part of your current Gartner subscription.

Container Supply Chain: 10 Security Vulnerabilities and How to Address Them

Solution Path for Increasing Application Agility Using Microservices Principles

MASA: Create Agile Application Architecture With Composable Apps, APIs and Services

Reference Architecture Brief: API Management

Reference Architecture Brief: API Access Control

Reference Architecture Brief: Event-Driven Application Architecture

Reference Architecture Brief: Microservices Architecture Delivery Platform

Reference Architecture Brief: Cloud-Native Application Architecture

This research note is restricted to the personal use of .

Guidance Framework for Deploying Containers and Kubernetes

When to Use a Service Mesh in Cloud-Native Architectures

How to Evaluate API Management Solutions

How a Service Mesh Fits Into Your API Mediation Strategy

© 2025 Gartner, Inc. and/or its affiliates. All rights reserved. Gartner is a registered trademark of Gartner, Inc. and its affiliates. This publication may not be reproduced or distributed in any form without Gartner's prior written permission. It consists of the opinions of Gartner's research organization, which should not be construed as statements of fact. While the information contained in this publication has been obtained from sources believed to be reliable, Gartner disclaims all warranties as to the accuracy, completeness or adequacy of such information. Although Gartner research may address legal and financial issues, Gartner does not provide legal or investment advice and its research should not be construed or used as such. Your access and use of this publication are governed by Gartner's Usage Policy. Gartner prides itself on its reputation for independence and objectivity. Its research is produced independently by its research organization without input or influence from any third party. For further information, see "Guiding Principles on Independence and Objectivity." Gartner research may not be used as input into or for the training or development of generative artificial intelligence, machine learning, algorithms, software, or related technologies.

This research note is restricted to the personal use of .

Table 1: Workloads for API Mediation

Ideal	Possible	Inappropriate
■ Policy enforcement ■ Usage monitoring ■ Traffic management	■ Protocol mediation (e.g., converting SOAP to REST) ■ Format translation (e.g., converting XML to JSON) ■ Simple payload modification (e.g., filtering sensitive fields out of objects) ■ Simple orchestration	■ Transformation that involves business logic ■ Content enrichment from another service ■ Complex orchestration (e.g., invoking multiple inner APIs to construct a response to a single inbound call) ■ Complex payload modification

Source: Gartner

This research note is restricted to the personal use of .